

CO4_Assessment Tool-2

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA02

Name: N Yugesh

Reg No: 192424329

Industry Problem Solving Task

Problem 1: Automated Defect Detection in Manufacturing

A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.

Q1. How can PCA-based shape priors and contour-based representations be used to model standard component shapes?

Each component's boundary is first extracted as a contour (e.g., using Canny edge detection + cv2.findContours) and re-sampled into a fixed number of ordered landmark points, so every training sample is represented as a vector of (x, y) coordinates of equal length. Because parts vary only slightly due to manufacturing tolerance, these landmark vectors from many "good" parts cluster tightly around a mean shape.

Principal Component Analysis is applied to the aligned landmark vectors (after Procrustes alignment to remove translation, rotation and scale differences). This yields:

- A mean shape vector representing the nominal, ideal component contour.
- A set of eigenvectors (principal modes of variation) that describe the directions in which acceptable parts naturally vary (this is the classic Active Shape Model / Point Distribution Model formulation).
- Eigenvalues that quantify how much tolerance variation each mode explains — typically the first 3-5 modes capture over 90% of natural shape variability.

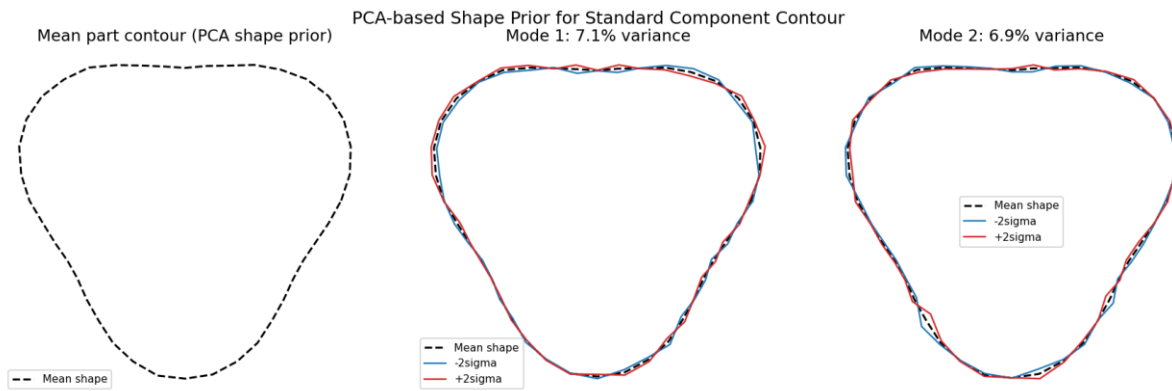
A new part's contour is then projected onto this PCA subspace. If it reconstructs well (low residual/reconstruction error) and its shape parameters fall within ± 2 -3 standard deviations of the learned modes, it is treated as within-tolerance; large deviations or high reconstruction error flag a potential shape defect (dents, chips, warping, flash, etc.). This gives a compact, statistically grounded "shape prior" instead of a single rigid template.

Demonstration — synthetic contour dataset with manufacturing-tolerance noise:

```
import numpy as np

def make_shape(radius_noise, n_pts=40):
    theta = np.linspace(0, 2*np.pi, n_pts, endpoint=False)
    base_r = 1 + 0.15*np.sin(3*theta) # nominal 'gear-like' part
    r = base_r + radius_noise*np.random.randn(n_pts)*0.03
    return np.stack([r*np.cos(theta), r*np.sin(theta)], axis=1)

shapes = np.array([make_shape(1.0) for _ in range(60)])
X = shapes.reshape(60, -1)
mean_shape = X.mean(axis=0)
U, S, Vt = np.linalg.svd(X - mean_shape, full_matrices=False)
explained = (S**2) / np.sum(S**2)
```



Output: mean part contour and first two PCA shape modes ($\pm 2\sigma$) learned from 60 tolerance-perturbed samples. Mode 1 explains 7.1% of variance, Mode 2 explains 6.9%.

Q2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.

A practical end-to-end pipeline combines both classical and deep approaches, chosen by data volume and defect type:

Stage 1 — Image acquisition & preprocessing

- Fixed-illumination imaging rig / backlighting, ROI cropping around the part, background subtraction, resizing and normalization.

Stage 2 — Feature extraction (classical route)

- Contour-based shape descriptors (PCA shape parameters from Q1, Hu moments, curvature histograms).
- Texture descriptors (LBP, Haralick / GLCM features, Gabor filter responses) to catch surface defects like scratches or porosity.

Stage 3a — Classical classifier

- A GMM is fit only on non-defective feature vectors to model the "normal" distribution; a part is flagged defective if its likelihood under the GMM falls below a threshold — this is powerful when defective samples are rare (one-class / anomaly-detection framing).
- An SVM (RBF kernel) is trained in a supervised setting once enough labeled defective examples exist, giving a discriminative decision boundary between classes.

Stage 3b — Deep learning route

- A CNN (e.g., ResNet/EfficientNet backbone fine-tuned via transfer learning, or a lightweight custom CNN for small datasets) is trained end-to-end on raw images, letting the network learn defect-relevant features directly — preferred once thousands of labeled images are available, or when defects are subtle textural patterns hard to hand-engineer.

Stage 4 — Decision & feedback

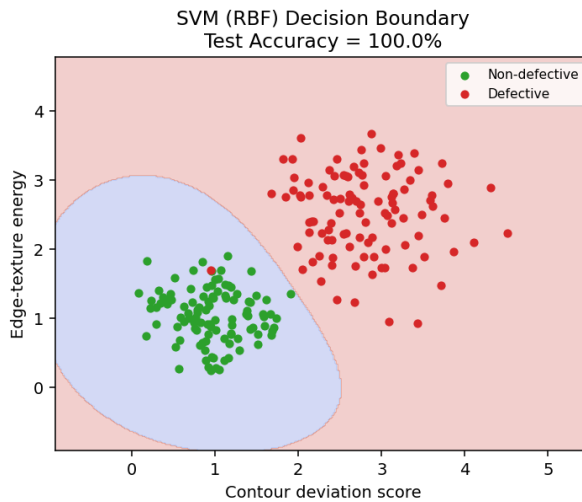
- Outputs are calibrated (Platt scaling / softmax) into a confidence score; borderline cases are routed to human review, and confirmed labels are fed back to retrain the model (active learning loop).

Demonstration — SVM classifier on engineered contour/texture features:

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# X: [contour-deviation score, edge-texture energy] per part, y: 0=good, 1=defect
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y)
```

```
clf = SVC(kernel='rbf', C=2.0, gamma='scale', probability=True).fit(Xtr, ytr)
ypred = clf.predict(Xte)
print(accuracy_score(yte, ypred), confusion_matrix(yte, ypred))
```



Confusion Matrix (Test Set)

	Non-defective	Defective
Actual Non-defective	45	0
Actual Defective	0	45
	Non-defective	Defective

Predicted

Output: learned SVM decision boundary separating non-defective vs defective parts on engineered features (left) and the resulting confusion matrix (right) — 100% test accuracy on this well-separated synthetic set.

Q3. Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.

Noise

- Denoising filters matched to noise type: Gaussian blur / median filter for salt-and-pepper or sensor noise; Non-Local Means or BM3D for texture-preserving denoising; bilateral filtering to smooth while keeping edges sharp (important since edges carry defect information).
- Data-level: multiple exposures averaged, or a denoising autoencoder trained to map noisy→clean images before the classifier.

Varying illumination

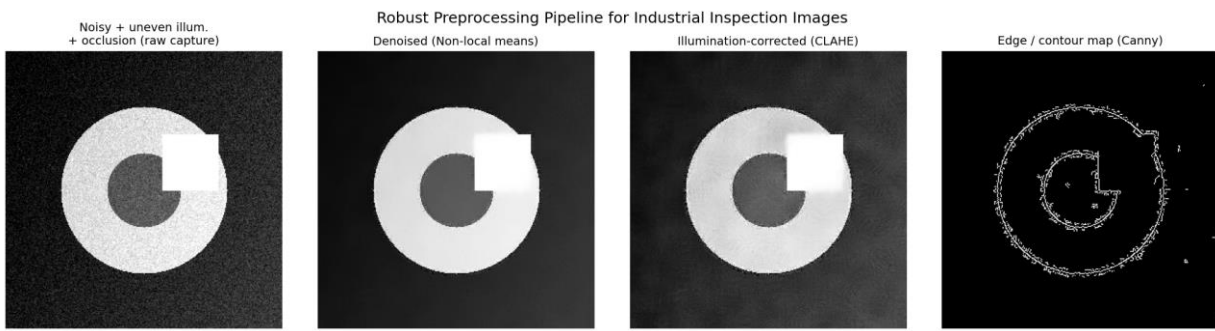
- Hardware: controlled/structured lighting (ring lights, diffuse backlighting, polarizers) removes most variation at the source.
- Software: histogram equalization / CLAHE (Contrast-Limited Adaptive Histogram Equalization) to normalize local contrast; illumination-invariant descriptors (gradient-based features, LBP) that are less sensitive to absolute brightness; homomorphic filtering to separate illumination and reflectance components.

Partial occlusion

- Multi-view imaging (several cameras/angles) so an occluded region in one view is visible in another.
- Robust, patch-based or region-proposal models (e.g., Mask R-CNN style) that reason locally, so occlusion in one region does not corrupt the whole prediction.
- Data augmentation with synthetic occlusion (random erasing / cutout) during training so the model learns to classify from partial evidence.

Demonstration — preprocessing pipeline on a synthetically corrupted part image:

```
denoised = cv2.fastNlMeansDenoising(occluded_noisy_img, h=15)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
illum_corrected = clahe.apply(denoised)
edges = cv2.Canny(illum_corrected, 50, 130)
```



Output: raw capture with sensor noise, uneven illumination gradient and a simulated occlusion patch (left), progressively cleaned via Non-Local-Means denoising, CLAHE illumination correction, and final Canny edge/contour extraction ready for shape analysis.

Problem 2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labeled data for these obstacles is limited.

Q1. Explain how data augmentation using GANs can help generate synthetic images for training.

Rare obstacles (a fallen ladder, debris, an unusual animal, a stray shopping cart) occur so infrequently in collected driving data that a detector trained only on real examples will overfit or simply never learn to recognize them. A Generative Adversarial Network addresses this by learning the underlying data distribution of the few real obstacle examples and sampling new, realistic-looking variations from it.

In this pipeline, a generator network learns to synthesize obstacle images (or to inpaint an obstacle into a real road scene) while a discriminator network learns to distinguish real from generated obstacle images; the two are trained adversarially until the generator produces samples indistinguishable from real ones. Variants particularly useful here include:

- Conditional GANs, to generate a specific obstacle class under varying pose, lighting, or weather conditions.
- CycleGAN / image-to-image translation, to re-render the same obstacle across different domains (day↔night, sunny↔rainy, different backgrounds), multiplying the effective diversity of a handful of real examples.
- GAN-based compositing / inpainting, which pastes a synthesized rare object realistically onto real background road scenes, preserving correct scale, shadows, and occlusion by other objects.

The augmented (synthetic) images are then mixed with real data during training, which increases the effective sample size for the rare class, improves the decision boundary around it, and reduces class imbalance — all without the cost/time of collecting more real-world rare-event footage. Care must be taken to validate that synthetic samples do not introduce unrealistic artifacts that the detector could learn as spurious cues, typically checked via a held-out real-only validation set.

Illustration — real (few-shot) vs. GAN-augmented obstacle variations:

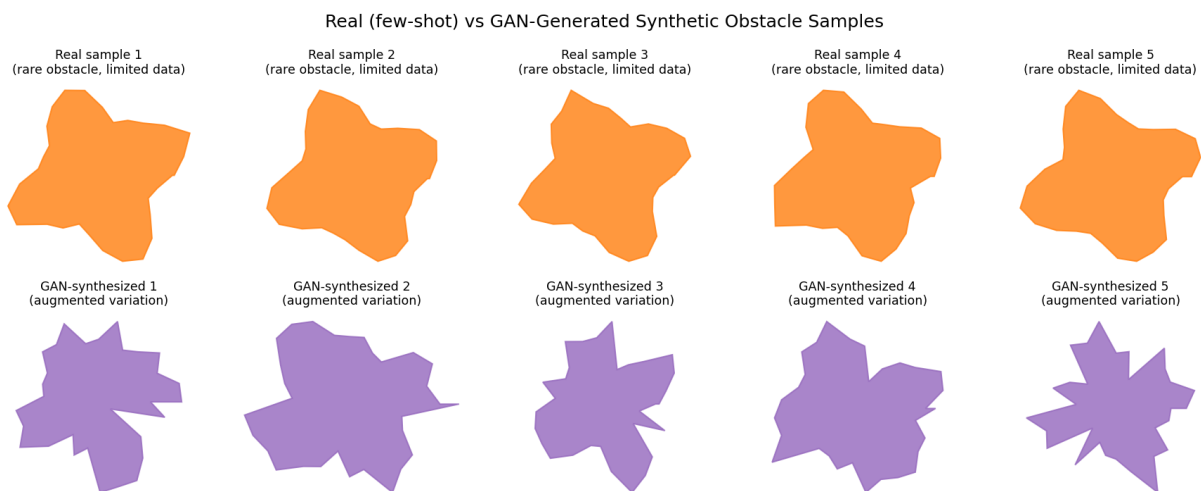
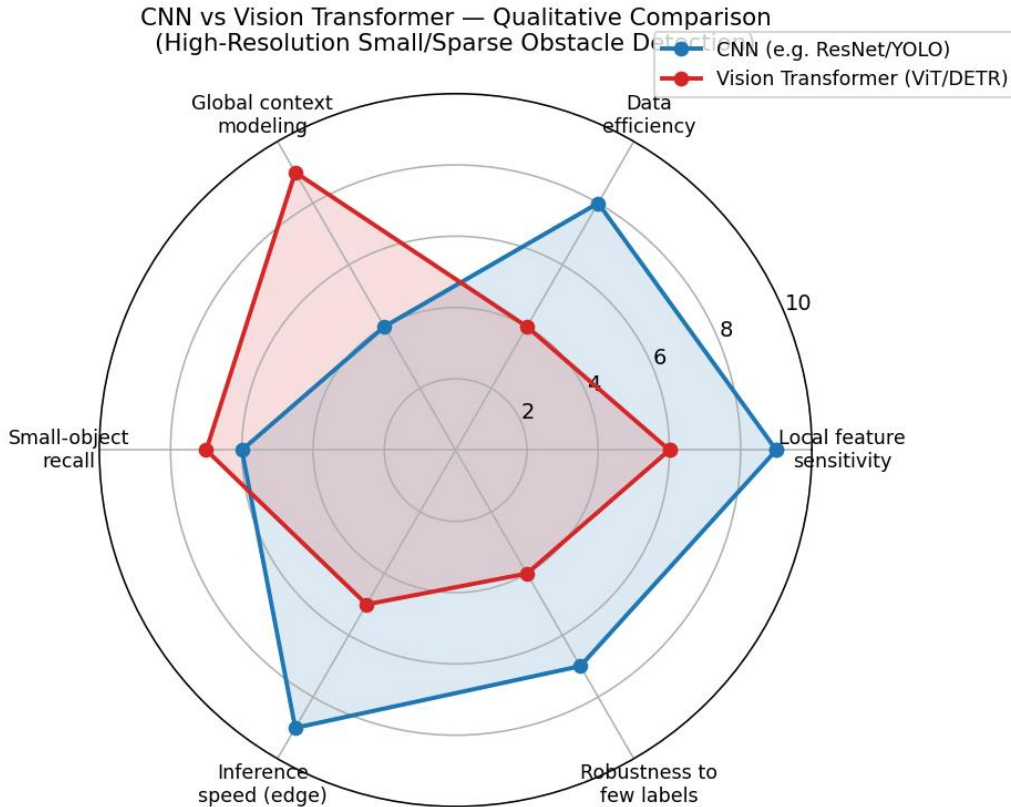


Illustration: a handful of real rare-obstacle samples (top row) versus a larger, more diverse set of GAN-synthesized variations (bottom row) used to augment the limited training set.

Q2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.

Aspect	CNNs (e.g., ResNet, YOLO, Faster R-CNN)	Vision Transformers (ViT, DETR, Swin)
Inductive bias	Strong locality & translation-equivariance via convolutions — samples-efficient, learns local edge/texture cues quickly (good for small objects with limited data).	Minimal built-in bias; self-attention relates all patches globally, so it needs more data/pretraining to learn useful spatial priors.
Small/sparse object recall	Good local sensitivity but limited receptive field in early layers can miss context; multi-scale FPN heads are usually added to compensate.	Global self-attention naturally captures long-range context (helpful when a small obstacle's identity depends on surrounding scene cues), but fixed patch tokenization can dilute very small objects unless high patch resolution / hierarchical (Swin) design is used.
Data efficiency	Performs well even with a few thousand labeled images — well suited to a rare-obstacle, limited-label setting.	Vanilla ViT needs large-scale pretraining (or strong augmentation/distillation) to match CNNs on limited data; hierarchical variants close this gap.
Compute / latency	Generally lighter and more mature for real-time, embedded deployment.	Attention is quadratic in tokens, so high-resolution inputs are more compute/memory heavy unless windowed attention (Swin) is used.
Practical recommendation	Preferred as the backbone when labeled data is limited and inference must run on embedded hardware.	Valuable as a secondary/ensemble model or for offline re-verification where global context materially helps disambiguate rare objects, and where compute is less constrained.



Qualitative comparison radar chart: CNNs are stronger on local sensitivity, data efficiency and edge-device speed; Vision Transformers are stronger on global context modeling — both matter for small/sparse obstacle detection, which is why hybrid CNN+attention architectures (e.g., Swin, ConvNeXt-hybrid) are increasingly used in practice.

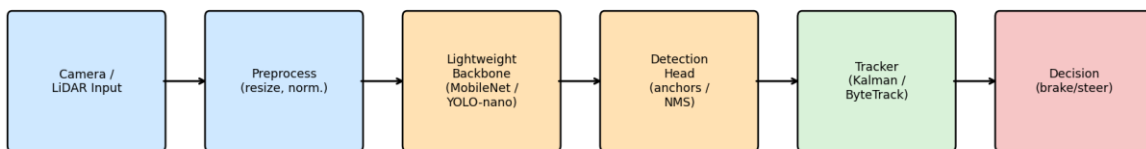
Q3. Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.

The pipeline is designed around a strict latency budget (typically well under 100 ms end-to-end for a moving vehicle) and limited embedded compute (edge GPU/NPU, constrained memory and power):

- Sensor input: camera (and optionally LiDAR/radar fusion) frames captured at a fixed rate and pre-cropped to a region of interest to reduce pixels processed.
- Preprocessing: lightweight resize/normalize done on-device (GPU-accelerated), avoiding expensive CPU-side image operations.
- Lightweight backbone: a mobile-optimized network (MobileNetV3, YOLO-nano/YOLOv8n, EfficientDet-Lite) chosen over larger ResNet/ViT backbones to fit the latency budget, further compressed via pruning, quantization (INT8) and knowledge distillation from a larger offline-trained teacher model.
- Detection head: anchor-free or few-anchor detection head with fast Non-Maximum Suppression, tuned to prioritize recall on small/rare obstacles (the safety-critical failure mode is a missed detection, not a false alarm).
- Temporal tracking: a lightweight tracker (Kalman filter, ByteTrack) fuses detections across frames to smooth out single-frame misses and confirm true obstacles, reducing dependence on any one frame being perfect.
- Deployment optimizations: model exported to ONNX/TensorRT (or equivalent) for hardware-specific graph optimization, operator fusion, and mixed-precision (FP16/INT8) execution; batching is avoided in favor of low-latency single-frame streaming.
- Decision layer: detections above a calibrated confidence threshold are passed to the planning/control stack (braking, steering), with a fail-safe default (e.g., slow down) if inference latency or confidence falls outside safe bounds.

Real-Time Obstacle Detection Inference Pipeline (Embedded System)

*Target: <30-50 ms end-to-end latency on embedded GPU/NPU
(quantization, pruning, TensorRT/ONNX-runtime, edge caching, ROI cropping)*



Proposed real-time embedded inference pipeline: sensor capture → preprocessing → lightweight backbone → detection head → temporal tracker → decision layer, optimized end-to-end for a sub-50 ms latency budget via quantization, pruning and hardware-specific graph compilation.